High CPU Utilization Analysis Using Thread Dump

Objective -

Identify the high CPU consuming process and thread, collect a thread dump, analyze the issue, and determine the root cause.

Environment Details -

Component	Details
OS	RHEL 8
App	OpenJDK17
User	appuser
Tools	top, jps, jstack

Incident Description   -

A High CPU issue was reported in java application . the system was running slowly due to high CPU usage .the issue needed to be investigated to identify the process & thread causing the problem

- **Investigation Steps –**

Step 1: Create Application User

useradd appuser

#passwd appuser

```
[appuser@srvcontroller highcpu_lab]$ id appuser
uid=1001(appuser) gid=1001(appuser) groups=1001(appuser)
[appuser@srvcontroller highcpu_lab]$ grep appuser /etc/passwd
appuser:x:1001:1001::/home/appuser:/bin/bash
[appuser@srvcontroller highcpu_lab]$
```

Observation : Application user verification and login successfully

Step 2 : Verify Java Installation

$ Java --version

$ Javac _- -version

$ Which jstack

```
[appuser@srvcontroller highcpu_lab]$ java --version
openjdk 17 2021-09-14
OpenJDK Runtime Environment 21.9 (build 17+35)
OpenJDK 64-Bit Server VM 21.9 (build 17+35, mixed mode, sharing)
[appuser@srvcontroller highcpu_lab]$ javac --version
javac 17
[appuser@srvcontroller highcpu_lab]$  which jstack
/usr/bin/jstack
```

Observation : Java 17 & jstack utility are available on the server

Step 3 : Create and Run Java Aapplication

$ Mkdir highcpu_lab

$ javac HighCPU.java

$ java HighCPU &

```
[appuser@ansiblecontroller ~]$ mkdir highcpu_lab
[appuser@ansiblecontroller ~]$ cd highcpu_lab/
[appuser@ansiblecontroller highcpu_lab]$ ls
[appuser@ansiblecontroller highcpu_lab]$ vim HighCPU.java
[appuser@ansiblecontroller highcpu_lab]$ javac HighCPU.java
[appuser@ansiblecontroller highcpu_lab]$ java HighCPU &
[1] 34026
[appuser@ansiblecontroller highcpu_lab]$ ps -ef | grep java
root        33800    2953 99 18:12 pts/0    00:03:38 java HighCPU
appuser     34026   33950 99 18:15 pts/0    00:00:14 java HighCPU
appuser     34046   33950  0 18:16 pts/0    00:00:00 grep --color=auto java
```

```java
public class HighCPU {
    public static void main(String[] args) {
        while(true) {
        }
    }
}
~
```

Observation: The Java application was created, compiled, and started successfully. The Java process was verified using the process list

Step 4 : Verify High CPU utilization

top

```
top - 19:38:40 up  1:45,  1 user,  load average: 1.00, 1.02, 1.00
Tasks: 310 total,   1 running, 309 sleeping,   0 stopped,   0 zombie
%Cpu(s): 50.2 us,  0.3 sy,  0.0 ni, 49.0 id,  0.0 wa,  0.5 hi,  0.0 si,  0.0 st
MiB Mem :   2774.3 total,    344.0 free,   1239.2 used,   1191.1 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   1351.0 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  34026 appuser   20   0 3291508  36288  25768 S  99.0   1.3  82:12.78 java
   2335 root      20   0  474340  81756  52340 S   0.7   2.9   0:08.92 Xorg
   2463 root      20   0 2705216 195200  98368 S   0.3   6.9   0:24.91 gnome-shell
   2946 root      20   0  715720  47472  32100 S   0.3   1.7   0:04.99 gnome-terminal-
      1 root      20   0  175712  14100   8832 S   0.0   0.5   0:05.10 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.04 kthreadd
```

top -H -p 34026

```
top - 19:39:49 up  1:46,  1 user,  load average: 1.04, 1.03, 1.00
Threads:  19 total,   2 running,  17 sleeping,   0 stopped,   0 zombie
%Cpu(s): 50.0 us,  0.0 sy,  0.0 ni, 49.8 id,  0.0 wa,  0.2 hi,  0.0 si,  0.0 st
MiB Mem :   2774.3 total,    341.9 free,   1241.4 used,   1191.1 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   1348.9 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  34027 appuser   20   0 3291508  36288  25768 R  99.7   1.3  83:18.89 java
  34026 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 java
  34028 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 GC Thread#0
  34029 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 G1 Main Marker
  34030 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 G1 Conc#0
  34031 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 G1 Refine#0
  34032 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.60 G1 Service
  34033 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.06 VM Thread
  34034 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 Reference Handl
  34035 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 Finalizer
  34036 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 Signal Dispatch
  34037 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 Service Thread
  34038 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.07 Monitor Deflati
  34039 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.02 C2 CompilerThre
  34040 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.04 C1 CompilerThre
  34041 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 Sweeper thread
  34042 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 Notification Th
  34043 appuser   20   0 3291508  36288  25768 R   0.0   1.3   0:02.26 VM Periodic Tas
  34044 appuser   20   0 3291508  36288  25768 S   0.0   1.3   0:00.00 Common-Cleaner
```

Observation :

Using the top command, the Java process (PID 34026) was identified as the high CPU consuming process. Further analysis using top -H -p 34026 showed that thread ID 34027 was consuming approximately 99% CPU.

Step 5 : convert thread ID to Hexadecimal

Printf ‘%x/n’ 34027

```
[root@ansiblecontroller highcpu_lab]# printf '%x\n' 34027
84eb
[root@ansiblecontroller highcpu_lab]#
```

Observation:

The high CPU thread ID was converted from decimal to hexadecimal format for thread dump analysis.

Step 6 : collect Thread Dump

Jstack 34026 > /threaddump.txt

ls -l /threaddump.txt

```
[root@ansiblecontroller highcpu_lab]# jstack 34026 > /threaddump.txt
[root@ansiblecontroller highcpu_lab]# ls -l /threaddump.txt
-rw-r--r--. 1 root root 4635 Jun 18 20:08 /threaddump.txt
[root@ansiblecontroller highcpu_lab]#
```

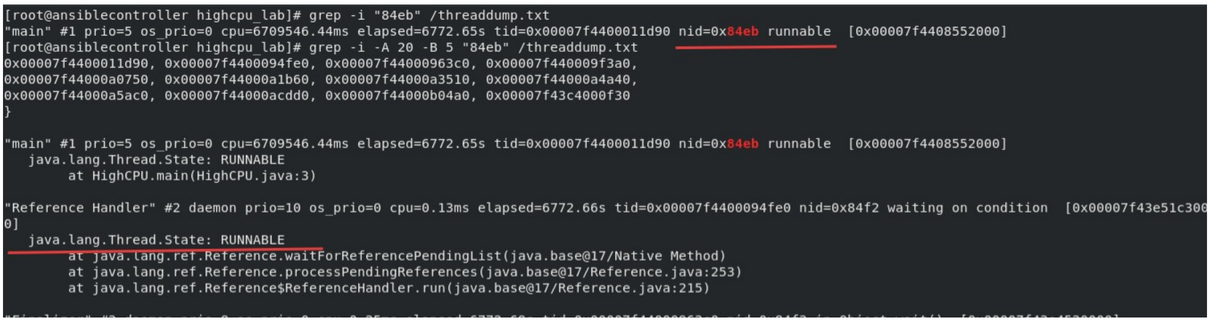
Observation:

Thread dump was collected successfully for further analysis.

Step 7 : analyze thread Dump

grep -i "84eb" threaddump.txt

grep -i -A 20 -B 5 "84eb" threaddump.txt



```
[root@ansiblecontroller highcpu_lab]# grep -i "84eb" /threaddump.txt
"main" #1 prio=5 os_prio=0 cpu=6709546.44ms elapsed=6772.65s tid=0x00007f4400011d90 nid=0x84eb runnable  [0x00007f4408552000]
[root@ansiblecontroller highcpu_lab]# grep -i -A 20 -B 5 "84eb" /threaddump.txt
0x00007f4400011d90, 0x00007f4400094fe0, 0x00007f44000963c0, 0x00007f440009f3a0,
0x00007f44000a0750, 0x00007f44000a1b60, 0x00007f44000a3510, 0x00007f44000a4a40,
0x00007f44000a5ac0, 0x00007f44000acdd0, 0x00007f44000b04a0, 0x00007f43c4000f30
}

"main" #1 prio=5 os_prio=0 cpu=6709546.44ms elapsed=6772.65s tid=0x00007f4400011d90 nid=0x84eb runnable  [0x00007f4408552000]
   java.lang.Thread.State: RUNNABLE
        at HighCPU.main(HighCPU.java:3)

"Reference Handler" #2 daemon prio=10 os_prio=0 cpu=0.13ms elapsed=6772.66s tid=0x00007f4400094fe0 nid=0x84f2 waiting on condition  [0x00007f43e51c300
0]
   java.lang.Thread.State: RUNNABLE
        at java.lang.ref.Reference.waitForReferencePendingList(java.base@17/Native Method)
        at java.lang.ref.Reference.processPendingReferences(java.base@17/Reference.java:253)
        at java.lang.ref.Reference$ReferenceHandler.run(java.base@17/Reference.java:215)
```

Observation:

The thread dump was analyzed using hexadecimal thread ID (0x84eb). The matching thread was identified in RUNNABLE state, indicating that it was actively consuming CPU resources.

Step 8 : Root Cause Analysis

RCA :

The Java application contained an infinite loop (while(true)), which caused the main thread to remain in RUNNABLE state and continuously consume CPU resources. As a result, the Java process utilized nearly 100% CPU, leading to high CPU utilization on the server.

Step 9 : Resolution

Kill -9 34026

Ps -ef | grep java

```
[root@ansiblecontroller highcpu_lab]# kill -9 34026
[root@ansiblecontroller highcpu_lab]# ps -el | grep java
[root@ansiblecontroller highcpu_lab]#
```

Observation:

The high CPU consuming Java process was terminated successfully. CPU utilization returned to normal levels after stopping the affected process.

Step 10: Conclusion

High CPU utilization was investigated using Linux monitoring tools and thread dump analysis. The problematic Java thread was identified, analyzed, and traced to an infinite loop in the application code. The issue was resolved by terminating the affected process.